Fermi–Dirac Statistics in a Semiconductor Crystal

Guhan Dhanasekharan (EP24BTECH11011), Kumud Jalori (EP24BTECH11015), Naman Verma (EP24BTECH11016), Nazakar Abhilash (EP24BTECH11017), Nithin Surya S (EP24BTECH11018), Panduru Sri Abish (EP24BTECH11019), Prasang Das (EP24BTECH11020), Praveen Kumar (EP24BTECH11021), S Haresh (EP24BTECH11022), and S Kshiteesh (EP24BTECH11023)
Department of Engineering Physics
(Dated: November 11, 2026)

We compute the electronic occupation statistics of a parabolic-band semiconductor by solving the carrier-density integral $n = \int D(E)f(E)\,dE$ numerically for the chemical potential. Writing the integral in terms of the dimensionless Fermi–Dirac integral $F_{1/2}$ keeps the units in one place and makes the integration cutoff a multiple of k_BT. The implementation is validated against both analytic limits: in the non-degenerate regime it reproduces not only the Boltzmann result but the leading Fermi–Dirac correction to it, to 3×10^{-5}; it reaches the degenerate $T \to 0$ Fermi energy to 10^{-7}; its temperature dependence follows the Sommerfeld expansion to 4×10^{-9}; and $F_{1/2}$ itself matches independent series to round-off over $-40 \le \eta \le 10^7$. An interactive Streamlit interface exposes temperature, carrier concentration, effective mass and band gap, reports E_F, k_F, v_F and T_F alongside a degeneracy flag, and warns whenever the inputs leave the range in which the single-band model is valid. We stress a distinction the usual phrasing obscures: the quantity that varies with temperature is the chemical potential $\mu(T)$, not the Fermi energy $E_F = \mu(0)$, and k_F, v_F and T_F are meaningful only when the carrier gas is degenerate — which at room temperature requires $n \gtrsim 10^{18}\,\mathrm{cm}^{-3}$.

I. INTRODUCTION

The electrons in a crystal are indistinguishable spin-$\frac{1}{2}$ fermions, so the Pauli principle limits each state to one of them. Their equilibrium distribution over energy is therefore not the Boltzmann distribution of classical statistics but the Fermi–Dirac distribution. In a metal this distinction is dramatic: the electron gas is degenerate, the occupation is essentially a step, and only electrons within k_BT of the Fermi level participate in transport or heat capacity. In a lightly doped semiconductor the carriers are dilute enough that Fermi–Dirac statistics reduce to Boltzmann statistics and the distinction nearly vanishes.

A semiconductor is interesting precisely because it can be moved between these two regimes by doping. This report computes where the boundary lies, by solving for the chemical potential without approximating the statistics, and uses that to examine how the Fermi level moves with temperature and carrier concentration.

II. THEORY

A. Fermi–Dirac statistics

The mean occupation of a single-particle state of energy E in equilibrium at temperature T is

$$f(E) = \frac{1}{\exp\big((E-\mu)/k_BT\big) + 1}, \tag{1}$$

where μ is the chemical potential. Two properties follow immediately and are used throughout. First, $f(\mu) = \frac{1}{2}$ at every temperature, so μ is the energy at which a state is equally likely to be occupied or empty. Second, as $T \to 0$, f becomes a step: every state below μ is filled, every state above is empty.

B. Density of states

In a cubic box of side L with periodic boundary conditions the allowed wavevectors form a lattice of spacing $2\pi/L$, so k-space contains $(L/2\pi)^3$ states per unit volume, doubled for spin. Counting states in a shell of radius k and converting to energy with the effective-mass dispersion $E = E_c + \hbar^2k^2/2m^*$ gives the conduction-band density of states per unit volume,

$$D(E) = \frac{1}{2\pi^2}\left(\frac{2m^*}{\hbar^2}\right)^{3/2}\sqrt{E - E_c}, \qquad E \ge E_c, \tag{2}$$

and zero inside the gap. The square-root edge is the three-dimensional signature: states become available gradually above the band edge rather than all at once. Figure 1 sets out the counting.

C. Carrier density fixes the chemical potential

The carrier concentration is the occupied density of states, integrated:

$$n = \int_{E_c}^{\infty} D(E)\,f(E)\,dE. \tag{3}$$

Given n, T and m^*, this is one equation in the single unknown μ. Substituting $x = (E - E_c)/k_BT$ and $\eta =$

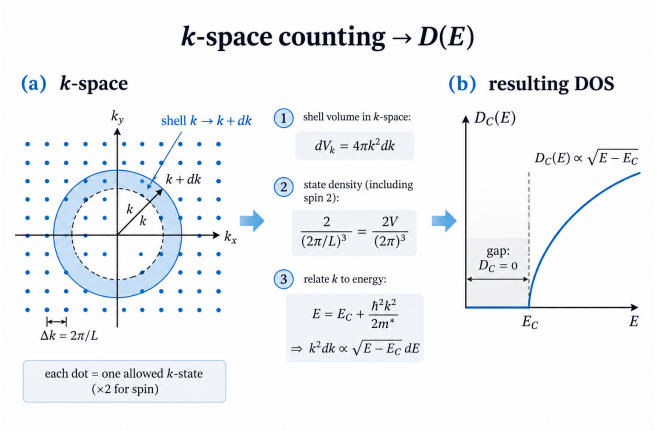


FIG. 1. Counting states in k-space to obtain $D(E)$. (a) Periodic boundary conditions put the allowed wavevectors on a lattice of spacing $\Delta k = 2\pi/L$; the sketch is a section through the three-dimensional grid, while the shell volume $4\pi k^2 dk$ in step 1 is the full three-dimensional one. Dividing that shell volume by the volume per state and doubling for spin gives the number of states it holds; step 3 converts dk to dE through the effective-mass dispersion. Carrying the constants through gives $D(E) = (1/2\pi^2)(2m^*/\hbar^2)^{3/2}\sqrt{E - E_c}$ per unit volume, the result quoted as Eq. (2). (b) The resulting band-edge behaviour, written $D_C(E)$ in the figure and $D(E)$ in the text: identically zero below E_c, rising with a vertical tangent at the edge and as $\sqrt{E - E_c}$ above it. Every later figure in this report inherits that square-root shape.

$(\mu - E_c)/k_BT$ reduces it to a dimensionless form,

$$n = N_c \frac{2}{\sqrt{\pi}} F_{1/2}(\eta), \qquad F_{1/2}(\eta) = \int_0^\infty \frac{\sqrt{x}\,dx}{1 + e^{\,x-\eta}}, \quad (4)$$

with the effective conduction-band density of states $N_c = 2\left(m^* k_B T/2\pi\hbar^2\right)^{3/2}$. This is the form we solve: it isolates all dimensional factors in N_c and makes the integration cutoff a multiple of k_BT rather than a fixed energy, which matters because the degenerate case extends far above the band edge.

D. The two limits

a. Non-degenerate ($\eta \ll -1$). The $+1$ in the denominator of (1) is negligible, $F_{1/2}(\eta) \to (\sqrt{\pi}/2)e^\eta$, and (4) collapses to the Boltzmann result

$$n = N_c \exp\left(\frac{\mu - E_c}{k_B T}\right). \quad (5)$$

The chemical potential lies *below* the band edge, there is no Fermi surface, and the carriers behave as a classical gas.

b. Degenerate ($\eta \gg 1$). Here $F_{1/2}(\eta) \to \frac{2}{3}\eta^{3/2}$ and (4) inverts to

$$E_F \equiv \mu(T{=}0) - E_c = \frac{\hbar^2}{2m^*}\left(3\pi^2 n\right)^{2/3}, \quad (6)$$

the familiar free-electron-gas result. The corresponding Fermi wavevector, velocity and temperature are

$$k_F = (3\pi^2 n)^{1/3}, \quad v_F = \frac{\hbar k_F}{m^*}, \quad T_F = \frac{E_F}{k_B}. \quad (7)$$

E. Two points of care

a. E_F is not $\mu(T)$. The two coincide only at $T = 0$. Equation (6) defines E_F as a function of n alone, with no temperature in it; the quantity that moves when the temperature changes is $\mu(T)$, obtained from (4). Where this report plots a "Fermi level versus temperature" it plots $\mu(T)$, and the distinction is not cosmetic: in Fig. 6(a) the $n = 10^{16}\,\mathrm{cm}^{-3}$ curve falls by 0.3 eV over the range shown while its E_F is fixed at 2.5 meV.

b. k_F, v_F and T_F presuppose degeneracy. They describe a filled Fermi sphere. For $n = 10^{16}\,\mathrm{cm}^{-3}$ in GaAs, Eq. (7) returns $T_F = 29$ K — far below room temperature, which is precisely the statement that the gas is *not* degenerate at 300 K and that these quantities are formal numbers rather than physical ones. The interface therefore reports them as asked and displays η beside them so the regime is never in doubt.

c. Multi-valley materials. For Si and Ge the electrons are shared between M_c equivalent ellipsoidal valleys ($M_c = 6$ and 4) with longitudinal and transverse masses m_l, m_t. Using the density-of-states mass $m_{de} = M_c^{2/3}(m_l m_t^2)^{1/3}$ in Eq. (6) gives the correct E_F, but Eq. (7) would put all n electrons in one sphere. Each valley actually holds n/M_c, so its mean Fermi radius is $(3\pi^2 n/M_c)^{1/3}$, smaller by $M_c^{1/3}$ (1.82 for Si), and the Fermi velocity is anisotropic, $v_i = \hbar k_i/m_i$ along each principal axis with $\hbar^2 k_i^2/2m_i = E_F$. For Si at $10^{20}\,\mathrm{cm}^{-3}$ this gives $k_F = 0.79$ rather than $1.44\ \mathrm{nm}^{-1}$, and $v_F = 1.6$ and 3.7×10^5 m/s along the long and short axes. The interface reports these per-valley values for the Si and Ge presets ($m_l, m_t = 0.98, 0.19$ and $1.64, 0.082$).

III. NUMERICAL METHOD

1. Evaluate $F_{1/2}(\eta)$ by adaptive quadrature with a purely relative tolerance of 10^{-12}, failing loudly if it is not met. The upper limit is $\max(\eta, 0) + 60$; the integrand falls off as $e^{-(x-\eta)}$, so this is far past negligible. Two details matter at the extremes. For $\eta < 0$ the factor e^η is taken outside the integral, so the quadrature works on an integrand of order one however small $F_{1/2}$ is; with an absolute tolerance instead, the error would rise to 10^{-3} for $\eta \lesssim -20$. For $\eta > 0$ the range is split at $x = \eta$, where the occupation steps from 1 to 0, and the part more than 60 below the step is integrated exactly, so the step is resolved even at $\eta \sim 10^5$.

2. Solve $n(\eta) = n_{\mathrm{target}}$ for η with Brent's method.

3. Bracket the root using the two closed forms, which bound it: the Boltzmann form overestimates n at a given η, so the root lies above $\ln(n/N_c)$, and the $T = 0$ sphere underestimates it, so the root lies below E_F/k_BT. A fixed bracket fails at low temperature, where η can reach 10^4 or more.

4. Return $\mu = E_c + \eta k_B T$.

Evaluating $f(E)$ through the logistic function `expit` avoids overflow when $|E - \mu| \gg k_BT$, which otherwise produces `inf` and a silent `nan`.

A. Validation

Nine checks run in `test_fermi.py`; all pass.

Check	Result
Non-degenerate: $\eta - \eta_{\text{Boltz}}$ vs $e^\eta/2^{3/2}$	3.2×10^{-5}
Degenerate $T \to 0$ limit vs Eq. (6)	9.6×10^{-8}
Carrier-density round trip	6.9×10^{-15}
$f(\mu) = \frac{1}{2}$	exact
Sommerfeld expansion for $\mu(T)$	3.6×10^{-9}
$D(E)$ zero in the gap, analytic above E_c	exact
$F_{1/2}$ vs series, $-40 \le \eta \le -0.5$	1.1×10^{-15}
$F_{1/2}$ vs asymptotic series, $10^3 \le \eta \le 10^7$	2.2×10^{-16}
Multi-valley E_F, k_F, valley volume	exact

TABLE I. Validation results, as maximum relative errors. The remaining check, not listed, confirms that $T = 0$ and $n = 0$ are rejected with an error rather than returning `nan`.

The first two rows deserve a comment, because their residuals are physics, not numerical error. Fermi–Dirac and Boltzmann statistics are not meant to agree exactly: $n = N_c\,(e^\eta - e^{2\eta}/2^{3/2} + \dots)$, so the solved η must exceed the Boltzmann value $\ln(n/N_c)$ by $e^\eta/2^{3/2}$ to leading order. The check is that it does, and the 3×10^{-5} is the size of the next-order term. Comparing η rather than μ also matters: a relative error in μ depends on where the zero of energy is placed. Likewise, at 1 K the Sommerfeld correction $(\pi^2/12)(k_BT/E_F)^2$ is itself 9.6×10^{-8}, so the second row is the physical offset of $\mu(1\,\text{K})$ from E_F.

The fifth is the strongest: in the degenerate regime the Sommerfeld expansion predicts

$$\mu(T) \simeq E_F \left[1 - \frac{\pi^2}{12}\left(\frac{k_BT}{E_F}\right)^2\right], \tag{8}$$

and the numerical solution follows this T^2 law to nine digits, as Fig. 6(b) shows. That tests the temperature dependence quantitatively rather than merely checking its sign.

IV. THE INTERACTIVE INTERFACE

The interface is written with Streamlit and launched with `streamlit run app.py`. The sidebar takes the band gap, effective mass, temperature, carrier concentration and energy range, with presets for GaAs, Si and Ge. The main panel reports $\mu - E_c$, E_F, k_F, v_F and T_F, states which regime the parameters fall in, and plots the band edges with $f(E)$ on a shared energy axis, $D(E)$ with its occupied part, and both sweeps. Every change recomputes the full integral; nothing is approximated for speed, though repeated inputs are cached. The energy grids carry a dense patch of points within $12\,k_BT$ of μ, so the Fermi step is resolved even at 1 K, where $k_BT = 86\,\mu$eV is far below the uniform grid spacing.

The regime banner calls the gas degenerate for $\eta > 2$ and non-degenerate for $\eta < -3$, where the Boltzmann approximation is within 2% (at $\eta = -2$ it is still 4.7% off). Because n is an input and the model has neither holes nor donor levels, some inputs describe states no real crystal can be in, and the interface says so instead of plotting them silently:

- *Valence band.* If μ falls within $3k_BT$ of E_v or below it, which happens for Si at 1000 K and $10^{12}\,\text{cm}^{-3}$, where μ is 0.51 eV below E_v, the chosen n is below the intrinsic density and an error is shown; below mid-gap a milder warning is shown.

- *Freeze-out.* For the presets, if the donor fraction that would be ionised, $1/[1 + 2(n/N_c)e^{E_d/k_BT}]$, drops below 90%, a warning notes that a real crystal with fixed doping would lose free carriers, with μ settling between E_d and E_c. For Si at $10^{16}\,\text{cm}^{-3}$ this triggers below 135 K.

- *Heavy doping.* The carrier-concentration slider stops at $10^{20}\,\text{cm}^{-3}$ and anything above $10^{19}\,\text{cm}^{-3}$ is flagged as outside the model. For GaAs a warning also appears once μ comes within $3k_BT$ of the L valley.

V. RESULTS

Unless stated otherwise the material is GaAs-like, $E_g = 1.42$ eV and $m^* = 0.067\,m_e$, at $T = 300$ K.

A. The three regimes

n (cm^{-3})	$\mu - E_c$ (meV)	η	E_F (meV)	T_F (K)
10^{16}	−97.3	−3.77	2.5	29
10^{18}	41.9	1.62	54.4	632
10^{19}	250.4	9.69	252.6	2931

TABLE II. GaAs at 300 K across three doping levels. Degeneracy requires $T \ll T_F$, which the last column makes explicit.

Table II locates the crossover. At $10^{16}\,\text{cm}^{-3}$ the chemical potential sits 97 meV *below* the band edge and

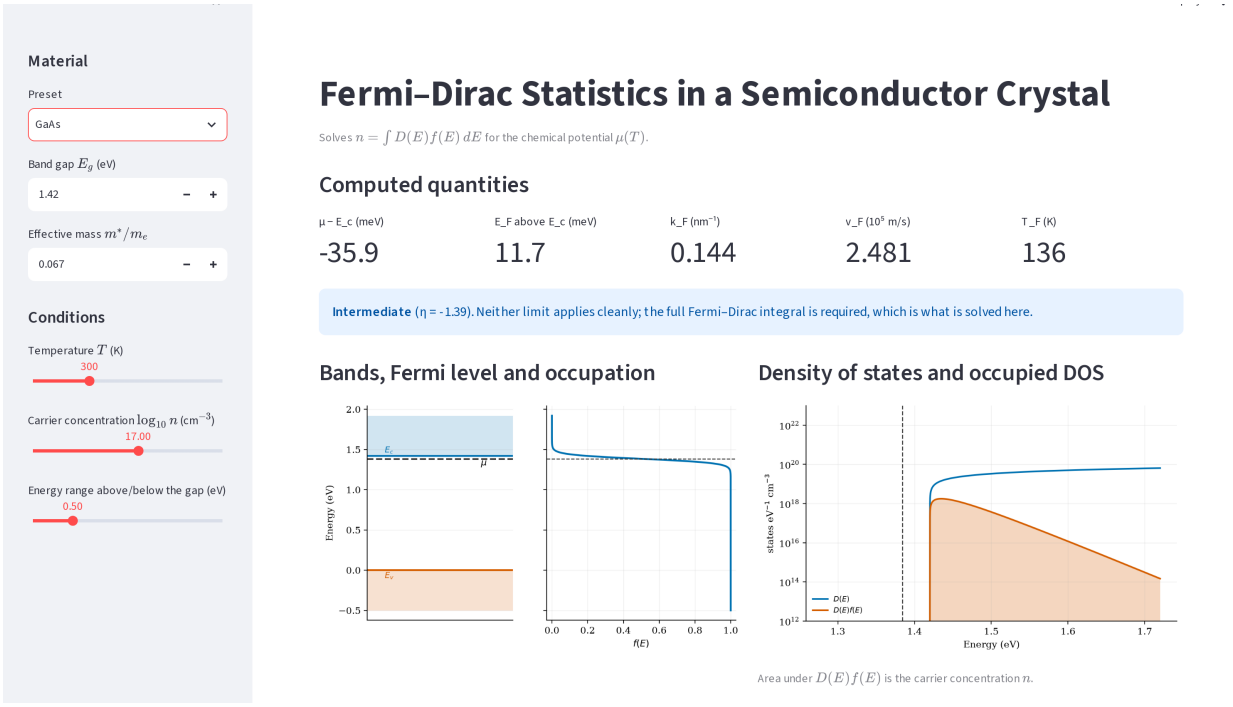


FIG. 2. The interface on the GaAs preset at $T = 300$ K and $n = 10^{17}\,\mathrm{cm}^{-3}$. The banner reports $\eta = -1.39$: neither limit applies cleanly here, which is why the full integral is solved rather than either closed form.

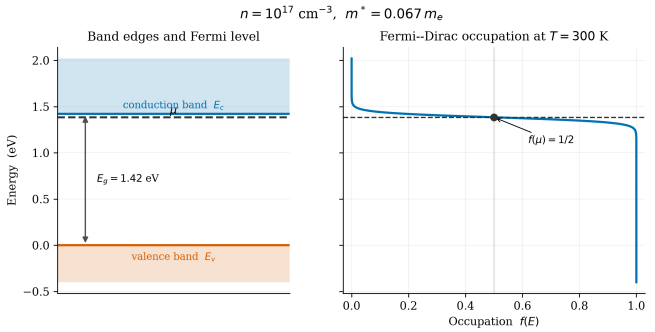


FIG. 3. Band edges, chemical potential and the Fermi–Dirac occupation on a shared energy axis, at $n = 10^{17}\,\mathrm{cm}^{-3}$. $f(E)$ passes through $\frac{1}{2}$ exactly at μ.

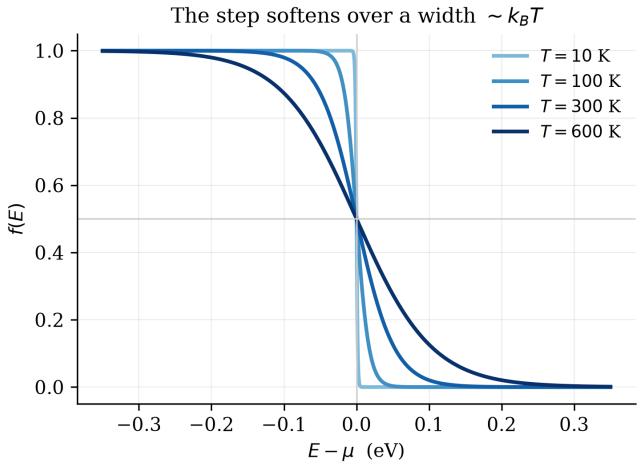


FIG. 4. The occupation step softens over a width of order $k_B T$. At 10 K it is nearly the $T = 0$ step; at 600 K it is smeared over more than 0.2 eV. The curves share a single reference μ, solved at 300 K and $10^{19}\,\mathrm{cm}^{-3}$ and then held fixed, so that the figure shows thermal broadening alone; the self-consistent $\mu(T)$ at fixed n moves as well, which is what Figure 6 plots.

$T_F = 29$ K is a tenth of the actual temperature: the gas is classical. At $10^{19}\,\mathrm{cm}^{-3}$, μ lies 250 meV *above* the edge and $T_F = 2931$ K, so 300 K is effectively a low temperature and μ has risen to within 1% of E_F. The case at $10^{18}\,\mathrm{cm}^{-3}$ sits between them, with $\eta = 1.62$: neither closed form is reliable there and the full integral earns its place.

a. Why the doping stops at 10^{19} cm^{-3}. It is tempting to push further to make the degenerate limit cleaner, but the model would stop describing the material. At $10^{20}\,\mathrm{cm}^{-3}$ Eq. (6) puts E_F at 1.17 eV above E_c, which is $0.83 E_g$ and lies far above the L and X satellite valleys of GaAs at roughly 0.29 and 0.48 eV. Those valleys would hold most of the carriers, so a single parabolic Γ valley is no longer the band structure and the computed E_F, while arithmetically correct, would be physically meaningless. Doping that heavy is also beyond what n-type GaAs achieves in practice, since Si dopants become amphoteric above roughly $10^{19}\,\mathrm{cm}^{-3}$. We there-

fore keep every reported case, and the range of Fig. 7, within $10^{19}\,\mathrm{cm}^{-3}$. Even that is the edge rather than the interior of the valid range: at $10^{19}\,\mathrm{cm}^{-3}$ and 300 K, μ is already within 40 meV of the L valley, so a small fraction of carriers would begin to spill into it.

Figure 5 shows the same crossover in the occupation itself. In the non-degenerate panel $D(E)f(E)$ falls exponentially above the band edge — a straight line on the logarithmic axis, the Boltzmann tail of Eq. (5). In the degenerate panel it instead tracks $D(E)$ itself right up to μ and then drops within a few k_BT: the states are simply filled to the chemical potential, as they would be in a metal.

B. Fermi level versus temperature

Figure 6(a) shows μ falling as temperature rises, and the reason is the shape of $D(E)$. The density of states grows as $\sqrt{E - E_c}$, so there are more states above μ than below it within the thermal window. Raising T promotes carriers upward into that larger reservoir, and μ must fall to keep the total fixed. At low doping it falls below the band edge entirely, which is the formal statement that the carriers have gone classical.

The degenerate case is far flatter, and Fig. 6(b) shows why: the shift is the T^2 Sommerfeld correction of Eq. (8), which stays small while $k_BT \ll E_F$. The numerical points follow the analytic curve closely up to about 300 K and then begin to separate, reaching a 0.5% discrepancy at 700 K where $k_BT/E_F \approx 0.24$. That departure is the expected behaviour of an asymptotic expansion, and seeing where it begins is more informative than a case chosen to hide it.

C. Fermi level versus carrier concentration

Figure 7 shows μ rising with doping, logarithmically at first and then more steeply. The logarithmic stretch is Eq. (5) rearranged, $\mu = E_c + k_BT\ln(n/N_c)$: in the non-degenerate regime the chemical potential depends only on the *ratio* n/N_c. The curve crosses E_c near $n \simeq N_c$, and beyond that the Pauli principle takes over — further carriers must occupy higher states, and μ climbs as $n^{2/3}$ from Eq. (6), far faster than logarithmically.

Raising the temperature shifts the crossing to higher n, because $N_c \propto T^{3/2}$: a hot semiconductor has more thermally accessible states and needs heavier doping before degeneracy sets in.

D. Different materials

Table III isolates the role of the effective mass. All three materials carry the same carrier concentration, yet GaAs is closest to degeneracy and silicon furthest from it. The band gap is irrelevant to this — it only sets where E_c sits — and the ordering follows m^* instead, through $N_c \propto (m^*)^{3/2}$. A light effective mass means few available states, so a given carrier density fills them further up and pushes μ toward the band edge. This is why GaAs, with the lightest electrons of the three, becomes degenerate at the lowest doping.

Material	E_g (eV)	m^*/m_e	$\mu - E_c$ (meV)	η
GaAs	1.42	0.067	−35.9	−1.39
Si	1.12	1.080	−145.8	−5.64
Ge	0.66	0.560	−120.3	−4.65

TABLE III. The three presets at $n = 10^{17}\,\mathrm{cm}^{-3}$ and 300 K. The masses are density-of-states masses, which already include the valley degeneracy (six X valleys in silicon, four L in germanium), so N_c must not be multiplied by it a second time. They are larger than the conductivity masses quoted for mobility calculations.

VI. CONCLUSIONS

Solving the carrier-density integral directly, rather than assuming either limit, gives a method that spans the classical and degenerate regimes with one code path and reproduces both closed forms where they apply: in the Boltzmann limit it recovers the leading Fermi–Dirac correction to 3×10^{-5}, and in the degenerate limit it reaches E_F to 10^{-7}, the size of the physical Sommerfeld offset at 1 K. Applying it:

- The classical–degenerate boundary sits near $n \sim N_c$, which for GaAs at 300 K is $4.35 \times 10^{17}\,\mathrm{cm}^{-3}$. Equivalently, degeneracy requires $T \ll T_F$.
- μ falls with temperature because $D(E) \propto \sqrt{E - E_c}$ offers more states above the chemical potential than below it; at low doping it drops below the band edge.
- In the degenerate regime that fall is the T^2 Sommerfeld correction, which the numerics reproduce until k_BT/E_F ceases to be small.
- μ rises logarithmically with n while non-degenerate and as $n^{2/3}$ once degenerate.
- Effective mass, not band gap, controls how readily a material becomes degenerate.

The model's limitations bound where it may be used. The band is a single isotropic parabolic valley. Using density-of-states masses makes N_c and E_F correct for the multi-valley materials; k_F and v_F are then computed per valley from the anisotropic masses, as described in Sec. II, but the carrier statistics still treat each valley as parabolic. GaAs has L and X valleys only 0.29 and 0.48 eV above Γ, and once E_F approaches them those

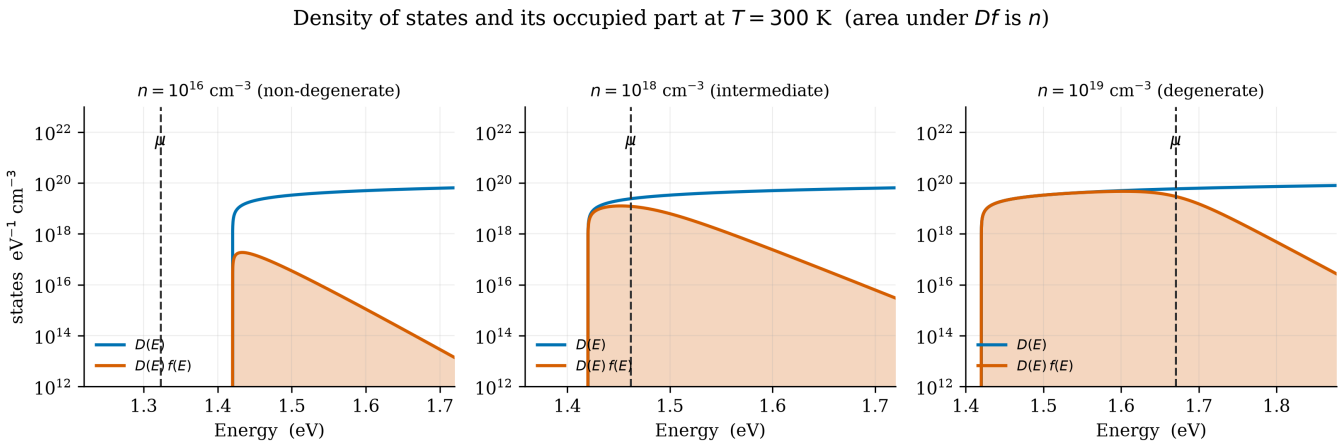


FIG. 5. $D(E)$ and the occupied density of states $D(E)f(E)$ across the three regimes, at 300 K. A logarithmic axis is necessary: at 10^{16} cm^{-3} the occupied curve lies some four orders of magnitude below $D(E)$ and would be indistinguishable from zero on a linear scale. The area under $D(E)f(E)$ is n in every panel. Both curves are the conduction-band quantities only; the valence band and holes lie outside the model. The occupied curve is clipped at 10^{10} m^{-3}eV^{-1} for plotting — a floor on the axis, not on the physics, which decays smoothly to zero.

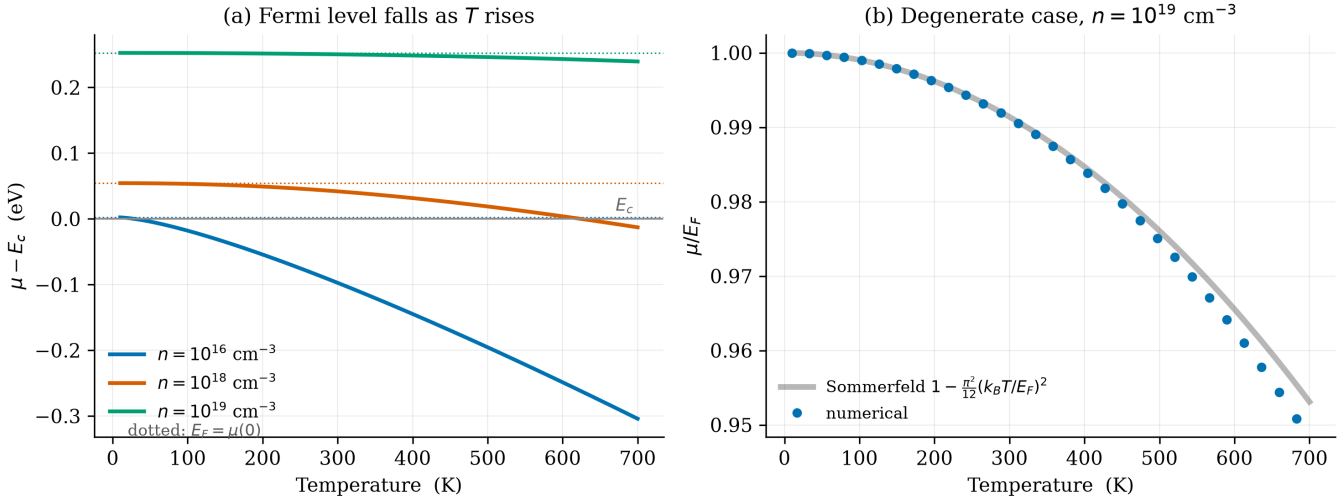


FIG. 6. (a) $\mu(T)$ at three carrier concentrations; dotted lines mark $E_F = \mu(0)$. (b) The degenerate case at 10^{19} cm^{-3} against the Sommerfeld expansion Eq. (8). The numerical points follow the analytic curve, departing from it by 0.5% only by 700 K, where k_BT/E_F is no longer small and the asymptotic expansion begins to fail as expected.

valleys hold carriers the model never counts — the reason the doping range above stops at 10^{19} cm^{-3}. Band non-parabolicity would reduce E_F further at high doping. We also neglect the valence band and dopant ionisation statistics, and treat n as an input rather than solving for it self-consistently from the dopant levels. Two consequences follow, and the interface flags both rather than hiding them. At low n and high T the fixed-n solution can place μ below E_v, a state that contradicts the model's own full valence band; with holes included, charge neutrality would pin μ near mid-gap instead. And as $T \to 0$ the solution sends μ to $E_c + E_F$ for any n, whereas a non-degenerately doped crystal freezes out: for Si at 10^{16} cm^{-3} and 5 K the model puts μ 0.75 meV below E_c, while freeze-out would hold it roughly 20 meV below. Solving $n - p = N_D^+$ with a valence band and a donor level would remove both limitations, and is the natural extension. The band gap is also held at its 300 K value; Varshni narrowing, about 0.24 eV for Si between 300 and 1000 K, would move E_v closer to μ in the first case.

REPRODUCING THIS WORK

```
uv venv --python 3.12 .venv
uv pip install -r requirements.txt
cd question_5
```

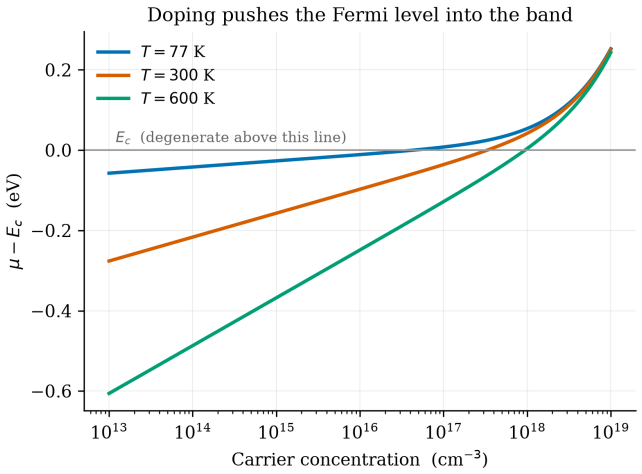


FIG. 7. $\mu - E_c$ against carrier concentration at three temperatures, up to the model's limit of $10^{19}\,\mathrm{cm}^{-3}$. The curve crosses into the band near $n \sim N_c$, which is $4.35 \times 10^{17}\,\mathrm{cm}^{-3}$ for GaAs at 300 K.

```
python test_fermi.py      # checks
python make_figures.py    # figures
streamlit run app.py      # interface
```

Appendix A: Complete source code

Listings are included directly from the source files, so they cannot drift from the code that produced the results above.

1. `fermi.py` — physics core

```python
"""Fermi-Dirac statistics for a parabolic-band semiconductor.

Single conduction band in the effective-mass approximation, 3D. Energies are
handled in joules internally; 'EV' converts to and from electronvolts.

Carrier density is evaluated through the dimensionless Fermi-Dirac integral

    n = N_c (2/sqrt(pi)) F_{1/2}(eta),    eta = (mu - E_c) / kT

which keeps the units in one place and makes the integration cutoff a multiple
of kT rather than a fixed energy.
"""

import numpy as np
from scipy import integrate, optimize
from scipy.special import expit

# ---- constants (SI) ----
HBAR = 1.054571817e-34
M_E = 9.1093837015e-31
K_B = 1.380649e-23
EV = 1.602176634e-19          # joules per electronvolt


# ---- occupation ----

def _check_positive(**kw):
    for name, v in kw.items():
        if not np.isfinite(v) or v <= 0:
            raise ValueError(f"{name} must be finite and positive, got {v!r}")


def fermi_dirac(E, mu, T):
    """f(E) = 1/(exp((E-mu)/kT) + 1), written to not overflow. Needs T > 0."""
    _check_positive(T=T)
    # expit(x) = 1/(1+exp(-x)), so f = expit(-(E-mu)/kT) with no large exp
    return expit(-(np.asarray(E, dtype=float) - mu) / (K_B * T))


```

```
# ---- density of states ----

def dos(E, m_eff, Ec=0.0):
    """Conduction-band DOS per unit volume per unit energy; zero below Ec."""
    E = np.asarray(E, dtype=float)
    out = np.zeros_like(E)
    above = E > Ec
    out[above] = (1.0 / (2 * np.pi**2)) * (2 * m_eff / HBAR**2) ** 1.5 \
        * np.sqrt(E[above] - Ec)
    return out


def n_c(m_eff, T):
    """Effective conduction-band density of states N_c."""
    _check_positive(m_eff=m_eff, T=T)
    return 2.0 * (m_eff * K_B * T / (2 * np.pi * HBAR**2)) ** 1.5


# ---- carrier density ----

def _quad(fn, a, b):
    """quad with a purely relative tolerance; fails loudly if it cannot meet it.

    The default epsabs (~1.5e-8) is an absolute floor, which for a tiny
    integral silently becomes a large relative error.
    """
    if b <= a:
        return 0.0
    val, err = integrate.quad(fn, a, b, epsabs=0.0, epsrel=1e-12, limit=400)
    if err > 1e-9 * abs(val):
        raise RuntimeError(f"Fermi integral did not converge (rel. err {err / abs(val):.1e})")
    return val


def fermi_integral_half(eta):
    """F_{1/2}(eta) = int_0^inf sqrt(x)/(1+exp(x-eta)) dx.

    eta < 0: the factor e^eta is taken outside, so the quadrature works on an
    O(1) integrand and the relative accuracy holds however small F is.
    eta >= 0: the range is split at x = eta, where the occupation steps from
    1 to 0, and again 60 below it, so the step is resolved even when eta is
    huge. The integrand decays like exp(-(x-eta)), so eta + 60 is far past
    negligible.
    """
    eta = float(eta)
    if eta < 0.0:
        # sqrt(x) expit(eta - x) = e^eta * sqrt(x) e^{-x} / (1 + e^{eta - x})
        g = lambda x: np.sqrt(x) * np.exp(-x) / (1.0 + np.exp(eta - x))
        return np.exp(eta) * _quad(g, 0.0, 60.0)
    h = lambda x: np.sqrt(x) * expit(eta - x)
    split = max(eta - 60.0, 0.0)
    # below `split` the occupation is 1 to within e^-60, so that piece is
    # exactly (2/3) split^{3/2} minus a negligible correction
    return (2.0 / 3.0) * split**1.5 + _quad(h, split, eta) + _quad(h, eta, eta + 60.0)


def carrier_density(mu, T, m_eff, Ec=0.0):
    """n = int D(E) f(E) dE, via the dimensionless integral."""
    eta = (mu - Ec) / (K_B * T)
    return n_c(m_eff, T) * (2.0 / np.sqrt(np.pi)) * fermi_integral_half(eta)


def solve_mu(n, T, m_eff, Ec=0.0):
    """Chemical potential mu(T) at fixed n, by root-finding on eta.

    The bracket comes from the two closed forms, which bound the root:
    Boltzmann overestimates n at a given eta (so the true eta is above
    ln(n/N_c)), and the T = 0 sphere underestimates it (so the true eta is
    below E_F/kT when that is positive). The bracket is then only widened if
    round-off makes those bounds touch the root.
    """
    _check_positive(n=n, T=T, m_eff=m_eff)
    target = float(n)
    pref = n_c(m_eff, T) * (2.0 / np.sqrt(np.pi))

    def residual(eta):
        # relative residual: same scale whatever n is
        return pref * fermi_integral_half(eta) / target - 1.0

    eta_b = np.log(target / n_c(m_eff, T))
    eta_d = fermi_quantities(target, m_eff)["E_F"] / (K_B * T)
    lo, hi = eta_b - 1.0, max(eta_d, eta_b) + 1.0
    r_lo, r_hi = residual(lo), residual(hi)
```

```
    for _ in range(60):
        if r_lo < 0.0 < r_hi:
            break
        if r_lo >= 0.0:
            lo -= max(10.0, abs(lo))
            r_lo = residual(lo)
        if r_hi <= 0.0:
            hi += max(10.0, abs(hi))
            r_hi = residual(hi)
    else:
        raise RuntimeError("could not bracket the chemical potential")

    eta = optimize.brentq(residual, lo, hi, xtol=1e-13, rtol=4 * np.finfo(float).eps)
    return Ec + eta * K_B * T


def boltzmann_mu(n, T, m_eff, Ec=0.0):
    """Non-degenerate closed form: n = N_c exp((mu-Ec)/kT). For validation."""
    _check_positive(n=n)
    return Ec + K_B * T * np.log(n / n_c(m_eff, T))


# ---- degenerate-gas quantities ----

def fermi_quantities(n, m_eff, valleys=None):
    """E_F (above Ec), k_F, v_F, T_F at T = 0 for carrier density n.

    Single isotropic valley (valleys=None): one Fermi sphere of mass m_eff.

    Multi-valley, valleys = (M_c, m_l, m_t) in kg: the carriers are shared
    between M_c equivalent ellipsoids, n/M_c each. E_F is the same as one
    sphere of density-of-states mass m_eff = M_c^{2/3} (m_l m_t^2)^{1/3}, but
    the Fermi surface is not: each valley has semi-axes k_l, k_t with
    hbar^2 k_i^2 / 2 m_i = E_F, and the velocity on it is hbar k_i / m_i
    along each principal direction. `k_F` is then the per-valley geometric
    mean radius (3 pi^2 n / M_c)^{1/3}; the directional values are k_F_l,
    k_F_t, v_F_l, v_F_t. `v_F` is sqrt(2 E_F / m_dv), a single-number summary
    only -- no electron on the surface need move at exactly that speed.

    These describe a degenerate gas. For a non-degenerate semiconductor they
    are formal numbers; `degeneracy` below says which regime applies.
    """
    _check_positive(n=n, m_eff=m_eff)
    if valleys is None:
        k_F = (3 * np.pi**2 * n) ** (1 / 3)
        E_F = HBAR**2 * k_F**2 / (2 * m_eff)
        return {"E_F": E_F, "k_F": k_F, "v_F": HBAR * k_F / m_eff, "T_F": E_F / K_B}

    M_c, m_l, m_t = valleys
    m_dv = (m_l * m_t**2) ** (1 / 3)          # density-of-states mass of ONE valley
    k_F = (3 * np.pi**2 * n / M_c) ** (1 / 3)
    E_F = HBAR**2 * k_F**2 / (2 * m_dv)
    k_l, k_t = (np.sqrt(2 * m * E_F) / HBAR for m in (m_l, m_t))
    return {"E_F": E_F, "k_F": k_F, "k_F_l": k_l, "k_F_t": k_t,
            "v_F_l": HBAR * k_l / m_l, "v_F_t": HBAR * k_t / m_t,
            "v_F": np.sqrt(2 * E_F / m_dv), "T_F": E_F / K_B}


def degeneracy(mu, T, Ec=0.0):
    """eta = (mu - Ec)/kT.  eta >> 1 degenerate, eta << -1 non-degenerate."""
    return (mu - Ec) / (K_B * T)


def sommerfeld_mu(n, T, m_eff, Ec=0.0):
    """Degenerate low-T expansion: mu ~ E_F[1 - (pi^2/12)(kT/E_F)^2]."""
    _check_positive(n=n, T=T)
    E_F = fermi_quantities(n, m_eff)["E_F"]
    return Ec + E_F * (1.0 - (np.pi**2 / 12.0) * (K_B * T / E_F) ** 2)
```

2. `test_fermi.py` — validation suite

```
"""Validation checks from PLAN.md. Run: python test_fermi.py"""

import numpy as np
import fermi as f

M = 0.067 * f.M_E          # GaAs-like effective mass
CM3 = 1e6                  # cm^-3 -> m^-3

```

```
9
10 def check(name, ok, detail=""):
11     print(f"[{'PASS' if ok else 'FAIL'}] {name}" + (f"  {detail}" if detail else ""))
12     return ok
13
14
15 # ---- 1. non-degenerate limit matches the Boltzmann closed form ----
16
17 # Compared in eta = (mu - Ec)/kT, which does not depend on where the energy
18 # zero sits (a relative error in mu would). Fermi-Dirac and Boltzmann are not
19 # meant to agree exactly: n = N_c (e^eta - e^{2 eta}/2^{3/2} + ...), so the
20 # solved eta must exceed ln(n/N_c) by e^eta/(2 sqrt 2) to leading order. The
21 # check is that the numerics reproduce that correction, not just a small gap.
22
23 def test_boltzmann():
24     worst_gap = worst_corr = 0.0
25     T = 300.0
26     for n_cm3 in (1e12, 1e14, 1e15):
27         n = n_cm3 * CM3
28         eta = f.degeneracy(f.solve_mu(n, T, M), T)
29         eta_b = f.degeneracy(f.boltzmann_mu(n, T, M), T)
30         corr = np.exp(eta_b) / (2 * np.sqrt(2))
31         worst_gap = max(worst_gap, abs(eta - eta_b))
32         worst_corr = max(worst_corr, abs((eta - eta_b) / corr - 1))
33     return check("non-degenerate vs Boltzmann, in eta", worst_gap < 1e-3,
34                  f"max |d eta| {worst_gap:.2e}") and \
35         check("eta - eta_Boltzmann equals the leading FD correction e^eta/2^1.5",
36               worst_corr < 1e-4, f"max rel dev {worst_corr:.2e}")
37
38
39 # ---- 2. degenerate limit: mu -> E_F as T -> 0 ----
40
41 def test_degenerate():
42     n = 1e19 * CM3
43     E_F = f.fermi_quantities(n, M)["E_F"]
44     mu = f.solve_mu(n, 1.0, M)          # 1 K is deep in the degenerate regime
45     err = abs(mu - E_F) / E_F
46     return check("degenerate T->0 vs E_F", err < 1e-4, f"rel err {err:.2e}")
47
48
49 # ---- 3. round trip: the solved mu reproduces n ----
50
51 def test_round_trip():
52     worst = 0.0
53     for n_cm3 in (1e14, 1e16, 1e18, 1e20):
54         for T in (50.0, 300.0, 600.0):
55             n = n_cm3 * CM3
56             back = f.carrier_density(f.solve_mu(n, T, M), T, M)
57             worst = max(worst, abs(back - n) / n)
58     return check("carrier-density round trip", worst < 1e-8, f"max rel err {worst:.2e}")
59
60
61 # ---- 4. f(mu) = 1/2 at any temperature ----
62
63 def test_half_occupancy():
64     worst = max(abs(f.fermi_dirac(0.3 * f.EV, 0.3 * f.EV, T) - 0.5)
65                 for T in (1.0, 77.0, 300.0, 1000.0))
66     return check("f(mu) = 1/2", worst < 1e-15, f"max dev {worst:.2e}")
67
68
69 # ---- 5. Sommerfeld expansion in the degenerate regime ----
70
71 def test_sommerfeld():
72     n = 1e20 * CM3                      # strongly degenerate, E_F ~ 1 eV
73     worst = 0.0
74     for T in (20.0, 50.0, 100.0):
75         num = f.solve_mu(n, T, M)
76         som = f.sommerfeld_mu(n, T, M)
77         worst = max(worst, abs(num - som) / abs(som))
78     return check("degenerate mu(T) vs Sommerfeld", worst < 1e-4,
79                  f"max rel err {worst:.2e}")
80
81
82 # ---- 6. DOS is zero in the gap and matches the analytic form above Ec ----
83
84 def test_dos():
85     Ec = 1.42 * f.EV
86     below = f.dos(np.linspace(0.0, Ec, 50), M, Ec)
87     E = Ec + np.array([0.01, 0.1, 0.5]) * f.EV
88     ana = (1 / (2 * np.pi**2)) * (2 * M / f.HBAR**2) ** 1.5 * np.sqrt(E - Ec)
89     return check("DOS zero in the gap", np.all(below == 0)) and \
90         check("DOS matches analytic form",
91               np.allclose(f.dos(E, M, Ec), ana, rtol=1e-12))
```

```python


# ---- 7. F_{1/2} itself, against independent closed forms at both extremes ----

def test_fermi_integral():
    # eta < 0: convergent series (sqrt(pi)/2) sum (-1)^{k+1} e^{k eta} / k^{3/2}
    k = np.arange(1, 400)
    worst_neg = max(abs(f.fermi_integral_half(eta) /
                        (np.sqrt(np.pi) / 2 * np.sum((-1) ** (k + 1) * np.exp(k * eta) / k**1.5))
                        - 1) for eta in (-40.0, -20.0, -15.0, -5.0, -0.5))
    # eta >> 1: Sommerfeld asymptotic series, truncation ~ eta^-6
    asym = lambda e: 2 / 3 * e**1.5 * (1 + np.pi**2 / (8 * e**2) + 7 * np.pi**4 / (640 * e**4))
    worst_pos = max(abs(f.fermi_integral_half(eta) / asym(eta) - 1)
                    for eta in (1e3, 1e5, 1e7))
    return check("F_1/2 vs series, eta in [-40, -0.5]", worst_neg < 1e-12,
                 f"max rel err {worst_neg:.2e}") and \
        check("F_1/2 vs asymptotic series, eta in [1e3, 1e7]", worst_pos < 1e-12,
              f"max rel err {worst_pos:.2e}")


# ---- 8. multi-valley Fermi quantities ----

def test_multivalley():
    # Si: 6 X valleys, m_l = 0.98, m_t = 0.19 -> m_de = 6^{2/3}(m_l m_t^2)^{1/3}
    Mc, ml, mt = 6, 0.98 * f.M_E, 0.19 * f.M_E
    m_de = Mc ** (2 / 3) * (ml * mt**2) ** (1 / 3)
    n = 1e20 * CM3
    one = f.fermi_quantities(n, m_de)
    many = f.fermi_quantities(n, m_de, valleys=(Mc, ml, mt))
    # E_F must agree with the equivalent sphere; k_F must not, by Mc^{1/3}
    ok_E = abs(many["E_F"] / one["E_F"] - 1) < 1e-12
    ok_k = abs(one["k_F"] / many["k_F"] - Mc ** (1 / 3)) < 1e-12
    # the ellipsoid's volume must hold n/Mc electrons: k_l k_t^2 = 3 pi^2 n/Mc
    ok_v = abs(many["k_F_l"] * many["k_F_t"] ** 2 / (3 * np.pi**2 * n / Mc) - 1) < 1e-12
    return check("multi-valley: E_F as one sphere, k_F smaller by M_c^1/3, "
                 "ellipsoid holds n/M_c", ok_E and ok_k and ok_v)


# ---- 9. unphysical input is rejected, not turned into nan / inf ----

def test_guards():
    bad = [lambda: f.fermi_dirac(0.0, 0.0, 0.0), lambda: f.solve_mu(1e20, 0.0, M),
           lambda: f.solve_mu(0.0, 300.0, M), lambda: f.boltzmann_mu(0.0, 300.0, M),
           lambda: f.sommerfeld_mu(0.0, 300.0, M)]
    raised = 0
    for call in bad:
        try:
            call()
        except ValueError:
            raised += 1
    return check("T = 0 and n = 0 raise ValueError", raised == len(bad),
                 f"{raised}/{len(bad)}")


if __name__ == "__main__":
    results = [test_boltzmann(), test_degenerate(), test_round_trip(),
               test_half_occupancy(), test_sommerfeld(), test_dos(),
               test_fermi_integral(), test_multivalley(), test_guards()]
    print(f"\n{sum(results)}/{len(results)} checks passed")
    raise SystemExit(0 if all(results) else 1)
```

3. make_figures.py — figure generation

```python
"""Regenerates every figure in figures/. Run: python make_figures.py"""

import os
import sys

import numpy as np
import matplotlib.pyplot as plt

sys.path.insert(0, os.path.join(os.path.dirname(__file__), "..", "common"))
import style
import fermi as f

FIG = os.path.join(os.path.dirname(__file__), "figures")

# GaAs-like reference material, energies measured from the valence-band edge
EG = 1.42 * f.EV
```

```
M = 0.067 * f.M_E
CM3 = 1e6
N_REF = 1e17 * CM3
T_REF = 300.0


def ev(E):
    return E / f.EV


# ---- band diagram with f(E) alongside ----

def fig_bands():
    mu = f.solve_mu(N_REF, T_REF, M, EG)
    E = np.linspace(-0.4 * f.EV, EG + 0.6 * f.EV, 2000)

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(9.5, 4.0), sharey=True,
                                   gridspec_kw=dict(width_ratios=[1.0, 1.3]))

    # left: the bands themselves
    ax1.axhspan(ev(EG), ev(E[-1]), color=style.ACOUSTIC, alpha=0.18, lw=0)
    ax1.axhspan(ev(E[0]), 0.0, color=style.OPTICAL[0], alpha=0.18, lw=0)
    ax1.axhline(ev(EG), color=style.ACOUSTIC, lw=2)
    ax1.axhline(0.0, color=style.OPTICAL[0], lw=2)
    ax1.axhline(ev(mu), color=style.EXACT, lw=1.8, ls="--")
    ax1.text(0.5, ev(EG) + 0.06, r"conduction band  $E_c$", ha="center", fontsize=9,
             color=style.ACOUSTIC)
    ax1.text(0.5, -0.16, r"valence band  $E_v$", ha="center", fontsize=9,
             color=style.OPTICAL[0])
    ax1.text(0.5, ev(mu) + 0.04, r"$\mu$", ha="center", fontsize=10)
    ax1.annotate("", xy=(0.12, ev(EG)), xytext=(0.12, 0.0),
                 arrowprops=dict(arrowstyle="<|-|>", color="0.3", lw=1.2))
    ax1.text(0.17, ev(EG) / 2, rf"$E_g = {EG/f.EV:.2f}$ eV", fontsize=9, va="center")
    ax1.set_xticks([])
    ax1.set_ylabel("Energy  (eV)")
    ax1.set_title("Band edges and Fermi level")

    # right: occupation on the same energy axis
    ax2.plot(f.fermi_dirac(E, mu, T_REF), ev(E), color=style.ACOUSTIC, lw=2)
    ax2.axhline(ev(mu), color=style.EXACT, lw=1.2, ls="--")
    ax2.axvline(0.5, color="0.8", lw=0.8)
    ax2.plot([0.5], [ev(mu)], marker="o", ms=6, color=style.EXACT, zorder=4)
    ax2.annotate(r"$f(\mu)=1/2$", xy=(0.5, ev(mu)), xytext=(0.62, ev(mu) - 0.28),
                 fontsize=9, arrowprops=dict(arrowstyle="->", lw=0.8))
    ax2.set_xlabel(r"Occupation  $f(E)$")
    ax2.set_xlim(-0.03, 1.03)
    ax2.set_title(rf"Fermi--Dirac occupation at $T={T_REF:.0f}$ K")

    fig.suptitle(rf"$n = 10^{{17}}$ cm$^{{-3}}$,  $m^*={M/f.M_E:.3f}\,m_e$", y=1.01)
    fig.savefig(f"{FIG}/bands.png")
    plt.close(fig)


# ---- f(E) sharpening as T falls ----

def fig_occupation():
    mu = f.solve_mu(1e19 * CM3, T_REF, M, EG)
    E = np.linspace(mu - 0.35 * f.EV, mu + 0.35 * f.EV, 2000)

    fig, ax = plt.subplots(figsize=(5.4, 3.6))
    cols = plt.cm.Blues(np.linspace(0.45, 1.0, 4))
    for T, c in zip([10.0, 100.0, 300.0, 600.0], cols):
        ax.plot(ev(E - mu), f.fermi_dirac(E, mu, T), color=c, label=f"$T={T:.0f}$ K")
    ax.axvline(0.0, color="0.8", lw=0.8)
    ax.axhline(0.5, color="0.8", lw=0.8)
    ax.set_xlabel(r"$E-\mu$  (eV)")
    ax.set_ylabel(r"$f(E)$")
    ax.set_title("The step softens over a width $\\sim k_BT$")
    ax.legend()
    fig.savefig(f"{FIG}/occupation.png")
    plt.close(fig)


# ---- D(E), f(E) and the occupied DOS ----

def dos_per_ev_cm3(E, m_eff, Ec):
    """DOS in the conventional semiconductor units: states eV^-1 cm^-3."""
    return f.dos(E, m_eff, Ec) * f.EV * 1e-6


def fig_dos():
    """D(E) and D(E)f(E) across three doping regimes.

```

```
    Log y: at n = 1e16 the occupied fraction is ~1e-7 of D(E), so on a linear
    axis the occupied curve would be a flat zero and show nothing.
    """
    fig, axes = plt.subplots(1, 3, figsize=(11, 3.5))

    for ax, (n_cm3, lab) in zip(axes, [(1e16, "non-degenerate"),
                                       (1e18, "intermediate"),
                                       (1e19, "degenerate")]):
        n = n_cm3 * CM3
        mu = f.solve_mu(n, T_REF, M, EG)
        # window follows mu, so the occupied region is bounded by physics
        # window must contain mu, which lies BELOW Ec when non-degenerate
        lo = min(EG - 0.02 * f.EV, mu - 4 * f.K_B * T_REF)
        hi = max(EG + 0.3 * f.EV, mu + 8 * f.K_B * T_REF)
        E = np.linspace(lo, hi, 3000)

        D = dos_per_ev_cm3(E, M, EG)
        occ = D * f.fermi_dirac(E, mu, T_REF)

        ax.semilogy(ev(E), D, color=style.ACOUSTIC, lw=1.8, label=r"$D(E)$")
        ax.semilogy(ev(E), np.maximum(occ, 1e10), color=style.OPTICAL[0], lw=1.8,
                    label=r"$D(E)\,f(E)$")
        ax.fill_between(ev(E), np.maximum(occ, 1e10), 1e10,
                        color=style.OPTICAL[0], alpha=0.25, lw=0)
        ax.axvline(ev(mu), color=style.EXACT, ls="--", lw=1.2)
        ax.annotate(r"$\mu$", xy=(ev(mu), 3e21), fontsize=10, ha="center")
        ax.set_ylim(1e12, 1e23)
        ax.set_xlim(ev(E[0]), ev(E[-1]))
        ax.set_xlabel("Energy  (eV)")
        ax.set_title(rf"$n=10^{{{int(np.log10(n_cm3))}}}$ cm$^{{-3}}$ ({lab})",
                     fontsize=9.5)
        ax.legend(fontsize=8, loc="lower left")

    axes[0].set_ylabel(r"states  eV$^{-1}$ cm$^{-3}$")
    fig.suptitle(rf"Density of states and its occupied part at $T={T_REF:.0f}$ K"
                 r"  (area under $D f$ is $n$)", y=1.03)
    fig.tight_layout()
    fig.savefig(f"{FIG}/dos_occupied.png")
    plt.close(fig)


# ---- mu versus temperature ----

def fig_mu_vs_T():
    """Two panels: the crossover below E_c, and the degenerate T^2 law."""
    T = np.linspace(10.0, 700.0, 160)
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 3.9))

    # (a) moderate doping -- mu drops below the band edge as T rises
    for n_cm3, c in zip([1e16, 1e18, 1e19],
                        [style.ACOUSTIC, style.OPTICAL[0], style.OPTICAL[1]]):
        n = n_cm3 * CM3
        mu = np.array([f.solve_mu(n, t, M, EG) for t in T])
        ax1.plot(T, ev(mu - EG), color=c,
                 label=rf"$n=10^{{{int(np.log10(n_cm3))}}}$ cm$^{{-3}}$")
        ax1.axhline(ev(f.fermi_quantities(n, M)["E_F"]), color=c, lw=0.8, ls=":")

    ax1.axhline(0.0, color="0.55", lw=1.0)
    ax1.text(695, 0.012, r"$E_c$", ha="right", fontsize=9, color="0.4")
    ax1.text(20, -0.33, r"dotted: $E_F=\mu(0)$", fontsize=8.5, color="0.35")
    ax1.set_xlabel("Temperature  (K)")
    ax1.set_ylabel(r"$\mu - E_c$  (eV)")
    ax1.set_title("(a) Fermi level falls as $T$ rises")
    ax1.legend(loc="lower left", fontsize=9)

    # (b) strongly degenerate -- the numerical mu follows the Sommerfeld law
    n = 1e19 * CM3
    E_F = f.fermi_quantities(n, M)["E_F"]
    Td = np.linspace(10.0, 700.0, 120)
    mu_num = np.array([f.solve_mu(n, t, M, EG) for t in Td])
    mu_som = np.array([f.sommerfeld_mu(n, t, M, EG) for t in Td])

    ax2.plot(Td, (mu_som - EG) / E_F, color=style.EXACT, lw=3.0, alpha=0.35,
             label=r"Sommerfeld $1-\frac{\pi^2}{12}(k_BT/E_F)^2$")
    ax2.plot(Td[::4], (mu_num[::4] - EG) / E_F, ls="none", marker="o", ms=4,
             color=style.ACOUSTIC, label="numerical")
    ax2.set_xlabel("Temperature  (K)")
    ax2.set_ylabel(r"$\mu / E_F$")
    ax2.set_title(rf"(b) Degenerate case, $n=10^{{19}}$ cm$^{{-3}}$")
    ax2.legend(loc="lower left", fontsize=8.5)

    fig.tight_layout()
    fig.savefig(f"{FIG}/mu_vs_T.png")
```

```
    plt.close(fig)


# ---- mu versus carrier concentration ----

def fig_mu_vs_n():
    # stops at 1e19, the model's stated limit (report Sec. IV): beyond it the
    # single parabolic Gamma valley no longer describes GaAs
    n_cm3 = np.logspace(13, 19, 200)
    fig, ax = plt.subplots(figsize=(5.8, 4.0))

    for T, c in zip([77.0, 300.0, 600.0], [style.ACOUSTIC, style.OPTICAL[0],
                                           style.OPTICAL[1]]):
        mu = np.array([f.solve_mu(x * CM3, T, M, EG) for x in n_cm3])
        ax.semilogx(n_cm3, ev(mu - EG), color=c, label=f"$T={T:.0f}$ K")

    ax.axhline(0.0, color="0.6", lw=1.0)
    ax.text(1.2e13, 0.02, r"$E_c$  (degenerate above this line)", fontsize=8.5,
            color="0.4")
    ax.set_xlabel(r"Carrier concentration  (cm$^{-3}$)")
    ax.set_ylabel(r"$\mu - E_c$  (eV)")
    ax.set_title("Doping pushes the Fermi level into the band")
    ax.legend(loc="upper left", fontsize=9)
    fig.savefig(f"{FIG}/mu_vs_n.png")
    plt.close(fig)


if __name__ == "__main__":
    style.apply()
    os.makedirs(FIG, exist_ok=True)
    fig_bands()
    fig_occupation()
    fig_dos()
    fig_mu_vs_T()
    fig_mu_vs_n()
    print("figures written to", os.path.abspath(FIG))
```

4. app.py — interactive interface

```
"""Streamlit interface for Fermi-Dirac statistics in a semiconductor.

Run:  streamlit run app.py
"""

import os
import sys

import numpy as np
import pandas as pd
import streamlit as st
from matplotlib.figure import Figure

_HERE = os.path.dirname(os.path.abspath(__file__))
# when run as a page of streamlit_app.py the entry script's directory is on
# sys.path, not this one, so add both explicitly. Streamlit re-runs this script
# on every widget change, so insert only once or the list grows without bound.
for _p in (os.path.normpath(os.path.join(_HERE, "..", "common")), _HERE):
    if _p not in sys.path:
        sys.path.insert(0, _p)
import style
import fermi as f

style.apply()
# only the entry script may configure the page; harmless no-op when this
# module runs as a page of streamlit_app.py
try:
    st.set_page_config(page_title="Fermi--Dirac Statistics in a Semiconductor", layout="wide")
except Exception:
    pass
st.title("Fermi\u2013Dirac Statistics in a Semiconductor Crystal")
st.caption(r"Solves $n=\int D(E)f(E)\,dE$ for the chemical potential $\mu(T)$.")

CM3 = 1e6


def ev(E):
    return E / f.EV


```

```
41
42  # ---- material data ----
43
44  # Eg (eV), density-of-states mass m_de/m_e, valleys (M_c, m_l, m_t) or None,
45  # shallow-donor ionisation energy E_d (eV), approximate Mott density (cm^-3).
46  # m_de already folds in the valley degeneracy (Si 6 X valleys, Ge 4 L), so N_c
47  # must not be multiplied by it again; m_de = M_c^{2/3} (m_l m_t^2)^{1/3}.
48  MATS = {
49      "GaAs": dict(Eg=1.42, m=0.067, valleys=None, Ed=0.0058, mott=1e16),
50      "Si":   dict(Eg=1.12, m=1.08, valleys=(6, 0.98, 0.19), Ed=0.045, mott=3.7e18),
51      "Ge":   dict(Eg=0.66, m=0.56, valleys=(4, 1.64, 0.082), Ed=0.0127, mott=3e17),
52  }
53  # above this the single parabolic valley stops describing any of the presets
54  # (satellite valleys in GaAs, non-parabolicity, dopant solubility): report Sec. IV
55  N_VALID = 1e19
56  # GaAs L valley above Gamma: carriers start spilling into it once mu nears it
57  GAAS_L = 0.29
58
59  # ---- inputs ----
60
61  with st.sidebar:
62      st.header("Material")
63      preset = st.selectbox("Preset", ["Custom", "GaAs", "Si", "Ge"])
64      mat = MATS.get(preset)
65      eg_d, me_d = (mat["Eg"], mat["m"]) if mat else (1.42, 0.067)
66
67      Eg = st.number_input("Band gap $E_g$ (eV)", 0.1, 6.0, float(eg_d), 0.01,
68                           key=f"eg_{preset}")
69      m_rel = st.number_input(r"Effective mass $m^*/m_e$", 0.01, 5.0, float(me_d),
70                              0.001, format="%.3f", key=f"me_{preset}")
71
72      st.header("Conditions")
73      T = st.slider("Temperature $T$ (K)", 1, 1000, 300)
74      log_n = st.slider(r"Carrier concentration  $\log_{10} n$ (cm$^{-3}$)",
75                        12.0, 20.0, 17.0, 0.1,
76                        help="The model is valid up to about 1e19 cm^-3; see the warning "
77                             "shown beyond that.")
78      span = st.slider("Energy range above/below the gap (eV)", 0.1, 2.0, 0.5, 0.1)
79
80  n = 10.0 ** log_n * CM3
81  m_eff = m_rel * f.M_E
82  Ec = Eg * f.EV
83  kT = f.K_B * T
84  # the valley structure belongs to the preset's mass; if the mass is edited the
85  # anisotropic masses no longer match it, so fall back to one isotropic valley
86  valleys = None
87  if mat and mat["valleys"] and abs(m_rel - mat["m"]) < 1e-9:
88      Mc, ml, mt = mat["valleys"]
89      valleys = (Mc, ml * f.M_E, mt * f.M_E)
90
91  # ---- solve ----
92
93
94  @st.cache_data(max_entries=256)
95  def mu_of(n, T, m_eff, Ec):
96      return f.solve_mu(n, T, m_eff, Ec)
97
98
99  @st.cache_data(max_entries=64)
100 def mu_sweep_T(n, m_eff, Ec):
101     Ts = np.linspace(5.0, 1000.0, 90)
102     return Ts, np.array([f.solve_mu(n, t, m_eff, Ec) for t in Ts])
103
104
105 @st.cache_data(max_entries=64)
106 def mu_sweep_n(T, m_eff, Ec):
107     ns = np.logspace(12, 20, 81)
108     return ns, np.array([f.solve_mu(x * CM3, T, m_eff, Ec) for x in ns])
109
110
111 mu = mu_of(n, float(T), m_eff, Ec)
112 eta = f.degeneracy(mu, T, Ec)
113 q = f.fermi_quantities(n, m_eff, valleys=valleys)
114
115 # ---- computed quantities ----
116
117 st.subheader("Computed quantities")
118 c1, c2, c3, c4, c5 = st.columns(5)
119 c1.metric("μ − E_c  (meV)", f"{ev(mu - Ec)*1000:.1f}")
120 c2.metric("E_F above E_c  (meV)", f"{ev(q['E_F'])*1000:.1f}")
121 if valleys:
122     c3.metric("k_F per valley  (nm$^{-1}$)", f"{q['k_F']/1e9:.3f}",
123               help=f"Geometric-mean radius of each of the {valleys[0]} valleys; "
```

```
                    f"semi-axes k_l = {q['k_F_l']/1e9:.3f}, k_t = {q['k_F_t']/1e9:.3f} nm$^{-1}$.")
    c4.metric("v_F  l / t  ($10^5$ m/s)", f"{q['v_F_l']/1e5:.2f} / {q['v_F_t']/1e5:.2f}",
              help="Fermi velocity along the long (l) and short (t) valley axes, "
                   "v_i = ħk_i/m_i. It is anisotropic, so no single value applies.")
else:
    c3.metric("k_F  (nm$^{-1}$)", f"{q['k_F']/1e9:.3f}")
    c4.metric("v_F  ($10^5$ m/s)", f"{q['v_F']/1e5:.3f}")
c5.metric("T_F  (K)", f"{q['T_F']:.0f}")

if eta > 2:
    st.success(f"**Degenerate** ($\eta$ = ($\mu$−E_c)/k_BT = {eta:.2f}). The Fermi surface is "
               "well defined, so $k_F$, $v_F$ and $T_F$ describe real carriers.")
elif eta < -3:
    st.warning(f"**Non-degenerate** ($\eta$ = {eta:.2f}). $\mu$ lies well below the band edge and "
               "Boltzmann statistics hold to better than 2 %. There is no Fermi surface, "
               "so $k_F$, $v_F$ and $T_F$ above are formal values, not physical ones.")
else:
    st.info(f"**Intermediate** ($\eta$ = {eta:.2f}). Neither limit applies cleanly (Boltzmann "
            "would be off by more than 2 %); the full Fermi--Dirac integral is required, "
            "which is what is solved here.")

if mat and mat["valleys"] and not valleys:
    st.caption("The mass differs from the preset's, so the valley structure is dropped "
               "and $k_F$, $v_F$ are for a single isotropic valley.")

# ---- where the model stops applying ----

if mu < 3 * kT:
    st.error(f"**Outside the model.** $\mu$ is {ev(mu)*1000:.0f} meV from the valence-band "
             "edge, within 3 k_BT of it or below it. The model has no holes, so it cannot "
             "describe this state: in a real crystal the intrinsic density at this "
             "temperature exceeds the chosen n, and $\mu$ would be pinned near mid-gap.")
elif mu < 0.5 * Ec:
    st.warning("$\mu$ is below mid-gap, so the chosen n is roughly below the intrinsic density "
               "at this temperature. Holes, which this model omits, would then outnumber "
               "electrons.")

if n / CM3 > N_VALID:
    st.warning(f"**Beyond the model's range** (n > $10^{19}$ cm$^{-3}$). A single parabolic valley "
               "no longer holds at this doping: satellite valleys fill, the band becomes "
               "non-parabolic and dopants stop being fully active. Treat the numbers as "
               "illustrative only.")
elif preset == "GaAs" and ev(mu - Ec) > GAAS_L - 3 * ev(kT):
    st.warning(f"$\mu$ is within a few k_BT of the GaAs L valley ({GAAS_L} eV above Γ), "
               "which would start to take carriers that this model does not count.")

if mat and n / CM3 < mat["mott"] and eta < 0:
    # fraction of donors ionised if n electrons came from donors at E_d below
    # E_c (non-degenerate, spin degeneracy 2): N_D+/N_D = 1/(1 + 2 (n/N_c) e^{E_d/kT})
    ion = 1.0 / (1.0 + 2.0 * n / f.n_c(m_eff, T) * np.exp(mat["Ed"] * f.EV / kT))
    ion_txt = "under 1 %" if ion < 0.01 else f"about {ion*100:.0f} %"
    if ion < 0.9:
        st.warning(f"**Freeze-out not modelled.** At {T} K only {ion_txt} of "
                   f"donors ({mat['Ed']*1000:.0f} meV below E_c) would be ionised. Here "
                   "n is held fixed, whereas a real crystal with fixed doping would lose "
                   "free electrons and $\mu$ would settle between the donor level and E_c.")
elif not mat and T < 100 and eta < 0:
    st.caption("At low temperature a real non-degenerate crystal freezes out (donors "
               "recapture electrons); this model holds n fixed and does not capture it.")

# ---- bands and occupation ----


def energy_grid(lo, hi, n_pts):
    """Uniform grid plus a dense patch around mu, so the Fermi step is resolved
    even when k_BT is far smaller than the uniform spacing."""
    patch = np.linspace(mu - 12 * kT, mu + 12 * kT, 600)
    E = np.concatenate([np.linspace(lo, hi, n_pts), patch[(patch > lo) & (patch < hi)]])
    return np.unique(E)


left, right = st.columns(2)
E = energy_grid(-span * f.EV, Ec + span * f.EV, 2000)

with left:
    st.subheader("Bands, Fermi level and occupation")
    fig = Figure(figsize=(7.2, 4.2))
    a1, a2 = fig.subplots(1, 2, sharey=True, gridspec_kw=dict(width_ratios=[1, 1.3]))
    a1.axhspan(ev(Ec), ev(E[-1]), color=style.ACOUSTIC, alpha=0.18, lw=0)
    a1.axhspan(ev(E[0]), 0.0, color=style.OPTICAL[0], alpha=0.18, lw=0)
    a1.axhline(ev(Ec), color=style.ACOUSTIC, lw=2)
    a1.axhline(0.0, color=style.OPTICAL[0], lw=2)
    a1.axhline(ev(mu), color=style.EXACT, lw=1.8, ls="--")
```

```
    a1.text(0.12, ev(Ec) + 0.04, "$E_c$", fontsize=9, color=style.ACOUSTIC)
    a1.text(0.12, -0.1, "$E_v$", fontsize=9, color=style.OPTICAL[0])
    # mu can sit within a few meV of Ec, so label it on the opposite side
    a1.text(0.78, ev(mu) - 0.085, r"$\mu$", fontsize=10)
    a1.set_xticks([])
    a1.set_ylabel("Energy (eV)")

    a2.plot(f.fermi_dirac(E, mu, T), ev(E), color=style.ACOUSTIC, lw=2)
    a2.axhline(ev(mu), color=style.EXACT, lw=1.0, ls="--")
    a2.set_xlabel("$f(E)$")
    a2.set_xlim(-0.03, 1.03)
    st.pyplot(fig)

with right:
    st.subheader("Density of states and occupied DOS")
    lo = min(Ec - 0.02 * f.EV, mu - 4 * kT)
    hi = max(Ec + 0.3 * f.EV, mu + 8 * kT)
    Ed = energy_grid(lo, hi, 2500)
    D = f.dos(Ed, m_eff, Ec) * f.EV * 1e-6
    occ = D * f.fermi_dirac(Ed, mu, T)

    fig = Figure(figsize=(7.2, 4.2))
    ax = fig.subplots()
    ax.semilogy(ev(Ed), np.maximum(D, 1e10), color=style.ACOUSTIC, lw=1.8,
                label="$D(E)$")
    ax.semilogy(ev(Ed), np.maximum(occ, 1e10), color=style.OPTICAL[0], lw=1.8,
                label="$D(E)f(E)$")
    ax.fill_between(ev(Ed), np.maximum(occ, 1e10), 1e10, color=style.OPTICAL[0],
                    alpha=0.25, lw=0)
    ax.axvline(ev(mu), color=style.EXACT, ls="--", lw=1.2)
    ax.set_ylim(1e12, 1e23)
    ax.set_xlabel("Energy (eV)")
    ax.set_ylabel(r"states eV$^{-1}$ cm$^{-3}$")
    ax.legend(loc="lower left", fontsize=8)
    st.pyplot(fig)
    st.caption("Area under $D(E)f(E)$ is the carrier concentration $n$.")

# ---- sweeps ----

st.subheader("Fermi level versus temperature and carrier concentration")
s1, s2 = st.columns(2)

with s1:
    Ts, mus = mu_sweep_T(n, m_eff, Ec)
    fig = Figure(figsize=(5.6, 3.4))
    ax = fig.subplots()
    ax.plot(Ts, ev(mus - Ec), color=style.ACOUSTIC, lw=2)
    ax.axhline(0.0, color="0.55", lw=1.0)
    ax.axhline(ev(q["E_F"]), color=style.EXACT, ls=":", lw=1.0)
    ax.plot([T], [ev(mu - Ec)], marker="o", ms=7, color=style.OPTICAL[0], zorder=4)
    ax.set_xlabel("Temperature (K)")
    ax.set_ylabel(r"$\mu-E_c$ (eV)")
    ax.set_title("dotted: $E_F=\\mu(0)$;  marker: current $T$", fontsize=9)
    st.pyplot(fig)
    st.caption("n is held fixed at every T, so the low-T end shows no donor freeze-out.")

with s2:
    ns, mun = mu_sweep_n(float(T), m_eff, Ec)
    fig = Figure(figsize=(5.6, 3.4))
    ax = fig.subplots()
    ax.axvspan(N_VALID, ns[-1], color=style.GAP_FILL, alpha=0.3, lw=0)
    ax.semilogx(ns, ev(mun - Ec), color=style.ACOUSTIC, lw=2)
    ax.axhline(0.0, color="0.55", lw=1.0)
    ax.plot([10.0 ** log_n], [ev(mu - Ec)], marker="o", ms=7,
            color=style.OPTICAL[0], zorder=4)
    ax.set_xlabel(r"$n$ (cm$^{-3}$)")
    ax.set_ylabel(r"$\mu-E_c$ (eV)")
    ax.set_title(r"above $E_c$ = degenerate;  shaded: beyond model", fontsize=9)
    st.pyplot(fig)

st.caption("Both sweeps are recomputed from the full Fermi--Dirac integral at every "
           "point; neither uses the Boltzmann approximation.")
```

---

[1] C. Kittel, *Introduction to Solid State Physics*, 8th ed., Wiley, 2005. Chapter 6, the free-electron Fermi gas.

[2] N. W. Ashcroft and N. D. Mermin, *Solid State Physics*, Holt–Saunders, 1976. Chapter 2 (Sommerfeld theory) and Chapter 28 (semiconductors).

[3] S. M. Sze and K. K. Ng, *Physics of Semiconductor Devices*,

3rd ed., Wiley, 2007. Chapter 1, carrier statistics.

[4] J. S. Blakemore, "Approximations for Fermi–Dirac integrals", *Solid-State Electronics* **25**, 1067 (1982).

[5] P. Virtanen *et al.*, "SciPy 1.0: fundamental algorithms for scientific computing in Python", *Nature Methods* **17**, 261–272 (2020).

[6] C. R. Harris *et al.*, "Array programming with NumPy", *Nature* **585**, 357–362 (2020).

[7] Streamlit documentation. https://docs.streamlit.io